Group 14 Technical Report

By Nicholas, Nathan, Yu-Shan

https://github.com/nicholassang/SE301-SVFRMS-Project

1. Legacy System Analysis

1.1 Purpose

The Smart Vertical Farming Resource Management System (SVFRMS) monitors the growing chambers of a vertical farm. Each chamber grows one crop, and the system checks the chamber's environment (temperature, humidity, soil moisture, light and nutrient levels) against that crop's requirements. When a reading is out of range, it raises an alert and activates a device, such as a cooling fan or water pump, to correct it. The system also tracks water and energy usage and produces a report at the end of each run.

1.2 Main Components

- All components are implemented inside a single class, SVFRMS.
- **Chamber data**: `ChamberState` holds each chamber's current readings, and `ChamberSummary` records its alert and action counts. Both are stored in shared static HashMaps.
- **Chamber monitoring**: four `ChamberThread` instances run at once, one per chamber (Tomato, Strawberry, Lettuce and Chili). Each performs 20 rounds of simulated readings for all five sensors.
- **Threshold evaluation**: `processSensor()` compares each reading against crop-specific limits, raises alerts and records resource costs when a limit is breached.
- **Event handling**: `handleEvent()` logs sensor readings, creates and prints alerts, and updates the chamber summaries.
- **Device control**: `addControlRequest()` places device requests on a shared queue (capacity 5). A dedicated `ControlThread` takes each request and calls `activateControl()`, which adjusts the chamber's readings for one of six devices: Cooling Fan, Water Pump, Nutrient Injector, Humidifier, Dehumidifier and Grow Light.
- **Resource tracking**: static counters record total water and energy usage.
- **Reporting and archiving**: after monitoring finishes, `generateDailyReport()` prints a summary, `archiveAlerts()` moves active alerts to an archive, and `saveArchiveReport()` writes the daily report to `archive/daily_report.txt`.
- **Lifecycle**: `main()` starts the control thread and the four chamber threads, waits for the chamber threads to finish, signals the control thread to stop, and then runs reporting and archiving.

2. Code Analysis and Limitations

2.1 Design Issues and Limitations

- **God class**: SVFRMS (393 lines) handles many responsibilities so every change touches the same class that holds the threads and locks.
    - Duplicated conditional logic: `processSensor()` has a separate if/else branch for every crop and sensor, each repeating the same alert, cost and queueing calls.

Adding a crop or device means adding more branches to existing methods.

- **String-based dispatch**: `handleEvent()` and `activateControl()` choose their behaviour by comparing strings such as "CREATE_ALERT" or "Cooling Fan". A misspelt string compiles without error and silently does nothing.

- **Global mutable state**: every collection, counter and lock is a public static field, so any part of the program can change any data.

2.2 Concurrency Issues

- **Race conditions**:
  - `totalWaterUsage`, `totalEnergyUsage`, `alerts`, `sensorLogs` are updated/written by four chamber threads. The sequence is not atomic, so updates are lost.
  - `getChamberState()` performs an unsynchronised check-then-act on a HashMap, so two threads can each create a state object and one is lost.

- **Synchronisation problems**:
  - `alertLock` is not taken where alerts are added, and `waterPumpLock` and `nutrientLock` are never used, so the locks protect nothing.
  - The report and archive threads are started together, so "Active Alerts" depends on which thread runs first.

- **Deadlock risk and liveness failure**:
  - `generateDailyReport()` and `archiveAlerts()` nest `alertLock` and `reportLock` in the same order, so no deadlock occurs today. But because the locks are public, any new method taking them in the opposite order would deadlock.

3. Proposed Design

The refactoring introduces modular components, clearer interfaces, and improved coordination between system operations. The main changes address separation of responsibilities, data management, configurable behaviour, and concurrency.

3.1 Separation of Responsibilities

System responsibilities are distributed across dedicated components:

- **Chamber management:** `ChamberRepository` manages chamber state, while `ChamberTask` handles individual chamber monitoring.
- **Sensor monitoring:** `SensorMonitor` processes readings and evaluates environmental conditions.
- **Alert management:** `SeverityPolicy` classifies each breach as INFO, WARNING or CRITICAL; `AlertManager` stores alerts, prevents duplicates and resolves recovered ones; `ArchiveManager` holds archived alerts.
- **Event recording:** `EventRecorder` records sensor readings, alerts, and control actions.

- **Resource tracking:** `ResourceUsageTracker` centralises resource consumption tracking.
- **Device control:** `ControlService` manages device operations and configurations.
- **Reporting:** `ReportManager` coordinates report generation and export.
- **Application coordination:** `SVFRMS` initialises components and manages execution.

3.2 Improved Data and Reporting Management

- **Repository abstraction:** `ChamberRepository` and `EventRepository` centralise data access and separate storage operations from business logic.
- **Consistent resource tracking:** `ResourceUsageTracker` provides a dedicated component for updating and retrieving resource consumption.
- **Report abstraction:** `Report` defines a common contract for different report types.
- **Data retrieval:** `ReportDataSource` separates report generation from the underlying data sources.
- **Output abstraction:** `ReportExporter` allows different export formats, with `TextReportExporter` as the current implementation.

Benefits:

- Reduces duplicated data access and reporting logic.
- Allows storage and export implementations to change independently.
- Supports adding report types without modifying existing report implementations.

3.3 Configurable System Behaviour

Hardcoded conditions and fixed parameters are replaced with configurable definitions where appropriate.

- **Crop configuration:** `CropProfile` and `Threshold` represent crop-specific environmental requirements.
- **Device configuration:** each Device is a name plus one or more `DeviceEffects`, each giving the sensor it adjusts, the step size and the limits. Devices are registered with `ControlService`.
- **Sensor configuration**: each `SensorType` carries its own default value, simulated range and severity limits.
- **Extensible registration:** `CropRegister` and the device registry in `ControlService` manage available configurations.
- **Sensor abstraction:** `SensorDataSource` separates sensor acquisition from monitoring logic, with `SimulatedSensorDataSource` providing the current implementation.

Benefits:

- Modify crop thresholds and device parameters without rewriting core processing logic.
- Introduce new crop profiles and devices through registration.
- Replace simulated sensor acquisition with another implementation of `SensorDataSource`.

3.4 Decoupled Control Execution

- `ControlRequest` encapsulates each requested device operation.
- `ControlQueue` holds pending requests.
- `ControlTask` consumes requests independently of the monitoring components.
- `ControlService` executes the requested operations and updates relevant chamber and resource state.

Benefits:

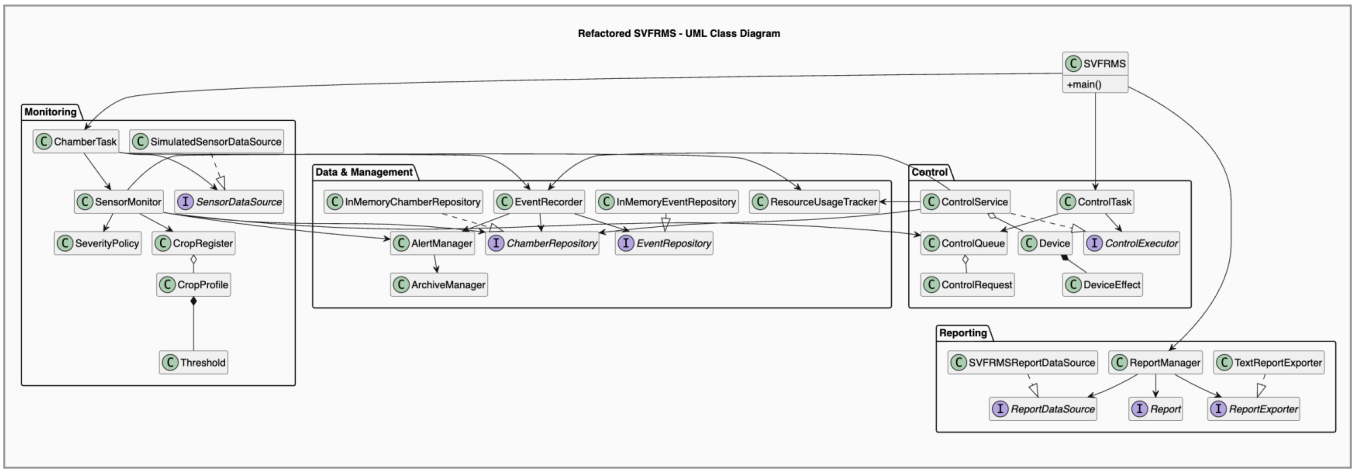
- Separates the decision to perform an action from its execution.
- Prevents monitoring tasks from directly managing actuator execution.
- Provides a centralised mechanism for processing control requests.

3.5 Concurrency and Execution Management

- `ChamberTask` implements `Runnable`, allowing multiple chambers to be monitored concurrently.
- `ExecutorService` manages worker threads and task execution.
- `ArrayBlockingQueue` bounds pending control requests and provides backpressure.
- A poison pill signals the control consumer to terminate gracefully.
- Synchronized methods protect shared chamber, alert, and resource state.
- `ConcurrentHashMap` supports concurrent access to the device registry.
- `CopyOnWriteArrayList` supports safe access to crop thresholds.
- Individual control request failures are handled without terminating the consumer.
- Alert archiving is completed before report generation to ensure consistent report contents.

4. Final Architecture

Figure 1. High-level component architecture of the refactored SVFRMS.



4.1 Main component interactions

- `ChamberTask` reads from `SensorDataSource` and passes each reading to `SensorMonitor`.
- `SensorMonitor` evaluates thresholds using `CropRegister`.

- `ChamberRepository` provides access to chamber state.
- `EventRecorder` records readings, alerts, and control actions.
- `AlertManager` manages alerts and delegates archiving.
- `SensorMonitor` submits control requests to `ControlQueue`.
- `ControlTask` processes requests through `ControlService`.
- `ReportManager` retrieves data through `ReportDataSource` and exports through `ReportExporter`.

4.2 Application execution sequence

1. Initialise system components.
2. Start the control consumer.
3. Execute chamber monitoring tasks concurrently.
4. Wait for chamber tasks to complete.
5. Stop the control consumer and await termination.
6. Archive alerts.
7. Generate and export reports.

5. Extensibility and Maintainability

The refactored design allows common changes to be made through configuration or interface implementations.

- **Add a crop:** Create a `CropProfile`, define its environmental thresholds, and register it through `CropRegister`. Existing monitoring logic can evaluate the new profile without adding crop-specific conditions.
- **Remove a crop:** Remove its profile from `CropRegister` and its chamber assignment from `chamberCrops`. Monitoring, control and reporting logic stays unchanged, and any chamber still assigned to a removed crop is skipped with a warning.
- **Modify thresholds:** Update the relevant `Threshold` configuration without changing `SensorMonitor`.
- **Add a report:** Implement the `Report` interface and register the new report with `ReportManager`. Existing report implementations remain unchanged.
- **Change report format:** Implement another `ReportExporter` and inject it into `ReportManager`, allowing output formats to change independently of report generation.
- **Replace simulated sensors:** Implement `SensorDataSource` for actual sensor readings without changing the monitoring component.
- **Add a control device:** Create a `Device` configuration and register it with `ControlService`, avoiding additional device-specific conditional logic.
- **Extend chamber management:** Add or modify chamber-related behaviour within the chamber management components without introducing unrelated responsibilities into `SVFRMS`.
- **Extend alert handling:** Modify alert lifecycle or archiving behaviour within `AlertManager` and its related components.
- **Extend event recording:** Introduce additional event types or recording behaviour through `EventRecorder` and the event storage abstraction.

- **Modify resource tracking:** Update resource accounting within `ResourceUsageTracker` without changing sensor monitoring or report implementations.
- **Change data storage:** Provide alternative repository or `ReportDataSource` implementations to change how system data is stored or retrieved.

Maintainability benefits:

- Localises changes within the responsible component.
- Reduces dependencies between business logic and implementation details.
- Enables testing with alternative or simulated dependencies.
- Encourages reuse through common interfaces.

6. Design Patterns

- **Strategy Pattern - `SensorDataSource`, `ReportExporter`:** Supports interchangeable sensor acquisition and report exporting implementations.
- **Repository Pattern - `ChamberRepository`, `EventRepository`:** Separates data access from business logic.
- **Producer–Consumer Pattern - `ControlQueue`, `ControlTask`:** Decouples control request submission from execution.
- **Thread Pool Pattern - `ExecutorService`:** Manages worker threads and their execution lifecycle.
- **Poison Pill Pattern - `ControlQueue`:** Signals the control consumer to terminate gracefully.
- **Registry Pattern - `CropRegister`, device registry in `ControlService`:** Centralises the registration and management of configurable entries.
- **Composition Root - `SVFRMS`:** Centralises component creation and dependency wiring, keeping application setup separate from business logic.
- **Data Transfer Object (DTO) - `ControlRequest`:** Carries control operation details between the request producer and consumer.

Object-Oriented Design Principles (SOLID)

- **Single Responsibility Principle (SRP):** Separates monitoring, alerting, event recording, resource tracking, control, and reporting into dedicated components.
- **Open/Closed Principle (OCP):** Supports extending reports, sensor sources, crop profiles, and device configurations without extensively modifying existing logic.
- **Liskov Substitution Principle (LSP):** Interface implementations such as `SensorDataSource` and `ReportExporter` can be substituted while preserving the expected behaviour of their consumers.
- **Interface Segregation Principle (ISP):** Uses focused interfaces such as `Report`, `ReportDataSource`, and `ReportExporter` to avoid unnecessary dependencies on unrelated operations.
- **Dependency Inversion Principle (DIP):** Uses abstractions to decouple high-level components from concrete implementations, supported by constructor-based dependency injection.